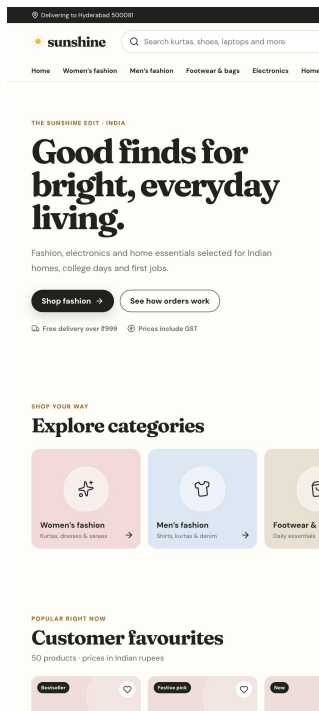


Sunshine

Agent-based Indian retail across frontend, Java backend and Python services



Author	Divya Rachala
Portfolio focus	Frontend + Java backend + Python backend/data
Project type	Final-year student-scale retail system
Region	India-first catalogue, pricing and delivery flow
Date	August 2026

A complete shopper journey from product discovery to a traceable order result. All products, payments and order data are synthetic demonstrations.

Project purpose and scope

Sunshine was designed to demonstrate a connected system rather than a collection of unrelated technologies.

Problem statement

A basic product grid does not show what happens after a shopper clicks buy. Sunshine models the complete decision path: stock reservation, payment outcome, delivery planning, customer feedback and a visible operational trace.

Portfolio objectives

- Build a clean, responsive Indian retail experience with prices in INR and familiar delivery expectations.
- Use Java for typed order orchestration and failure handling.
- Use Python for a FastAPI recommendation endpoint and a pandas retail KPI workflow.
- Demonstrate three bounded agents without misrepresenting them as generative AI.
- Deploy a working public demo while documenting the boundary between Vercel and local services.

Delivered scope

Area	Delivered capability	Evidence
Catalogue	50 synthetic products in five retail categories	Static data + 50 generated product pages
Customer flow	Search, category, product, cart, checkout and result	Browser-tested end-to-end paths
Java	Catalogue, Payment and Fulfilment agents	Spring Boot + three JUnit scenarios
Python	Recommendation API and retail KPI script	FastAPI + pandas + four pytest checks
Delivery	Docker Compose, CI and Vercel-ready UI	Three GitHub Actions jobs

The scope deliberately excludes authentication, a production database and a real payment gateway. Those choices keep the project explainable and credible for a final-year student.

From discovery to delivery

The interface follows a realistic retail sequence while keeping every payment and order operation safely simulated.

Step	Customer action	System response
1	Search or choose a category	Filters the 50-product catalogue
2	Open a product and choose size	Shows pricing, delivery and recommendations
3	Add item and adjust quantity	Persists cart in versioned local storage
4	Enter address and payment method	Validates Indian mobile and PIN formats
5	Choose a demonstration scenario	Runs success, decline or stock failure
6	Review order result	Shows order ID, totals and agent trace

Delivering to Hyderabad 500081



[Home](#)
[Women's fashion](#)
[Men's fashion](#)
[Footwear & bags](#)
[Electronics](#)
[Home](#)

SECURE DEMO CHECKOUT

Where should we send your order?

Use sample information only. This portfolio demo does not save your details.

1

Delivery address

Indian PIN codes are supported in this demo.

Full name

Mobile number

Address

City

PIN code

2

Payment method

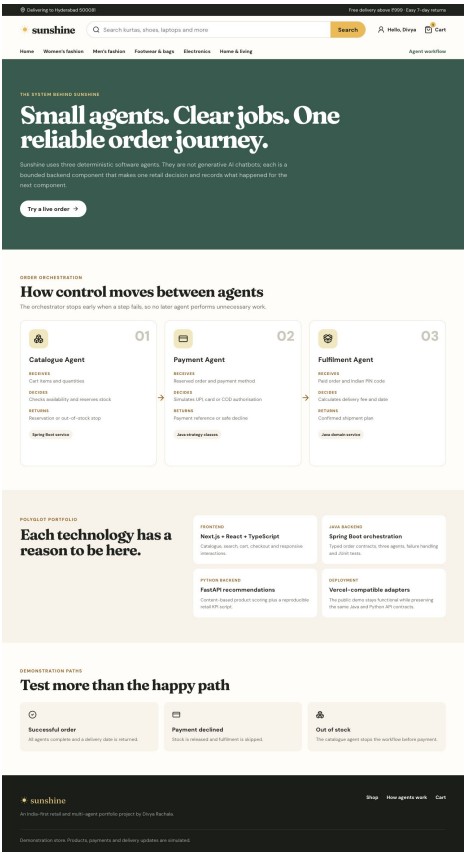
Retail rules

- Free delivery starts at INR 999; smaller carts add INR 79.
- UPI, card and cash on delivery are available as simulated methods.
- A successful order clears the cart; a declined order keeps it available for retry.

One browser contract, three technology stacks

The Next.js route handlers provide stable browser endpoints and proxy to Java or Python when their service URLs are configured.

Layer	Technology	Responsibility
Browser	React + TypeScript	Interactive catalogue, cart and checkout state
Web app	Next.js 16	Pages, metadata, route handlers and Vercel deployment
Order service	Java 17 + Spring Boot 4	Validation and three-agent orchestration
Recommendation service	Python 3.13 + FastAPI	Content-based product ranking
Analytics	pandas	Raw order CSV to KPI JSON
Delivery	Docker + GitHub Actions	Reproducible local stack and automated checks



Deployment adapter pattern

If `JAVA_BACKEND_URL` or `PYTHON_BACKEND_URL` is present, Next.js proxies to the real service. If a service is unavailable, a TypeScript adapter applies the same deterministic contract so the Vercel demo remains usable. The response identifies the source, which allowed integration tests to prove that Java and Python were actually called locally.

Bounded agents and explicit failure paths

Spring Boot is the canonical order service. The orchestrator calls agents in sequence and stops safely when a prerequisite fails.

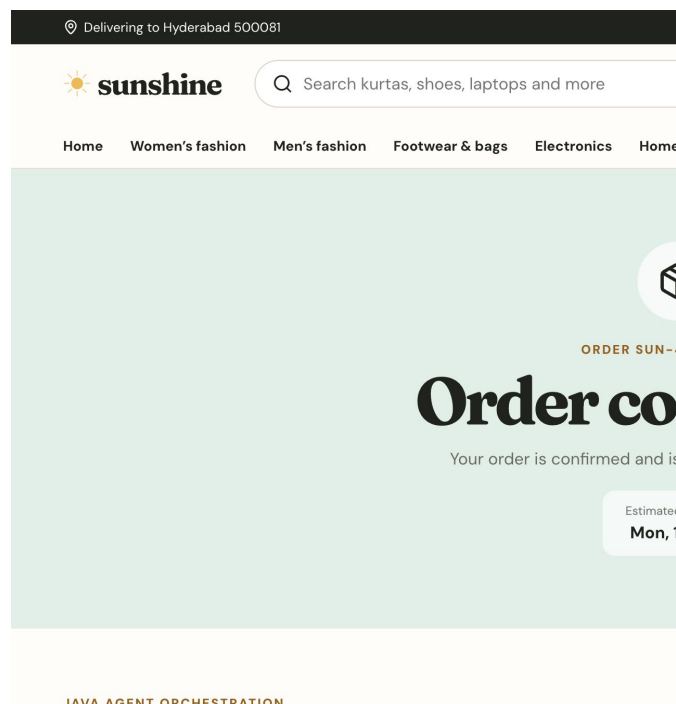
Agent	Input	Decision	Output
Catalogue	Cart lines	Can stock be reserved?	Reservation or OUT_OF_STOCK
Payment	Reserved order	Can the method be authorised?	Reference or PAYMENT_FAILED
Fulfilment	Eligible order + PIN	When can it arrive?	Delivery date and plan

Why agents instead of one large service?

Each class owns one reason to change. Inventory rules can evolve without changing delivery logic; payment methods can be extended without rewriting the catalogue check. The orchestrator remains small because it coordinates results rather than implementing every business rule.

Scenario behaviour

Scenario	Catalogue	Payment	Fulfilment
SUCCESS	completed	completed	completed
PAYMENT_FAILED	completed	failed	skipped
OUT_OF_STOCK	failed	skipped	skipped



The Java API returned the custom response header x-sunshine-service: java during local integration testing.

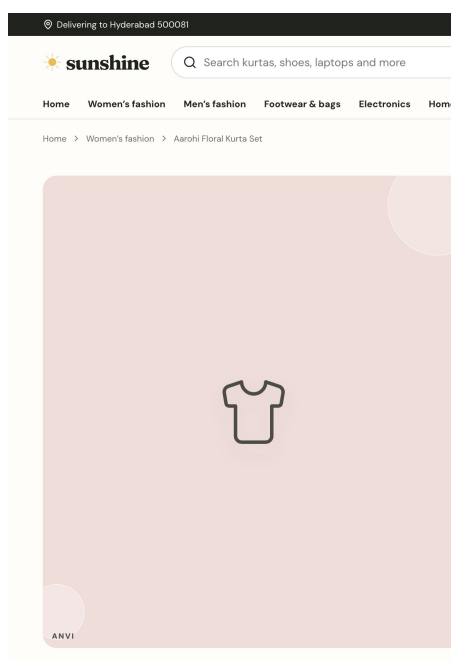
Recommendations and measurable retail KPIs

Python has two focused roles: ranking related products through FastAPI and transforming a raw order CSV through pandas.

Recommendation method

The service receives a reference product and same-contrast candidates from Next.js. It filters to the same category and ranks by rating and relative price distance. This is intentionally content-based because the demo does not collect customer histories.

$$\text{score} = \text{rating} \times 2 - \text{absolute price difference} / \text{reference price}$$

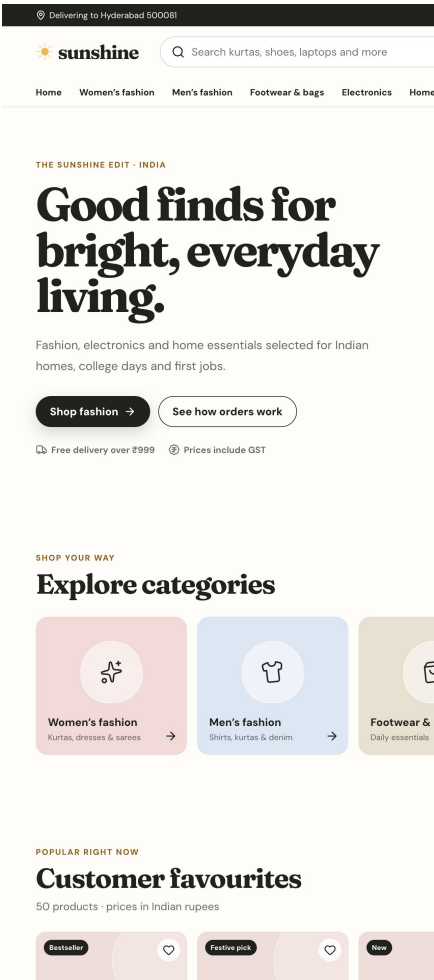


KPI pipeline output

KPI	Result
Orders received	12
Orders confirmed	9
Confirmation rate	75.0%
Confirmed revenue	INR 37,136
Average order value	INR 4,126.22
Top revenue city / category	Delhi / Electronics

Retail clarity without copying an existing brand

Sunshine borrows familiar retail information patterns but uses an original visual identity, synthetic products and CSS-built illustration cards.

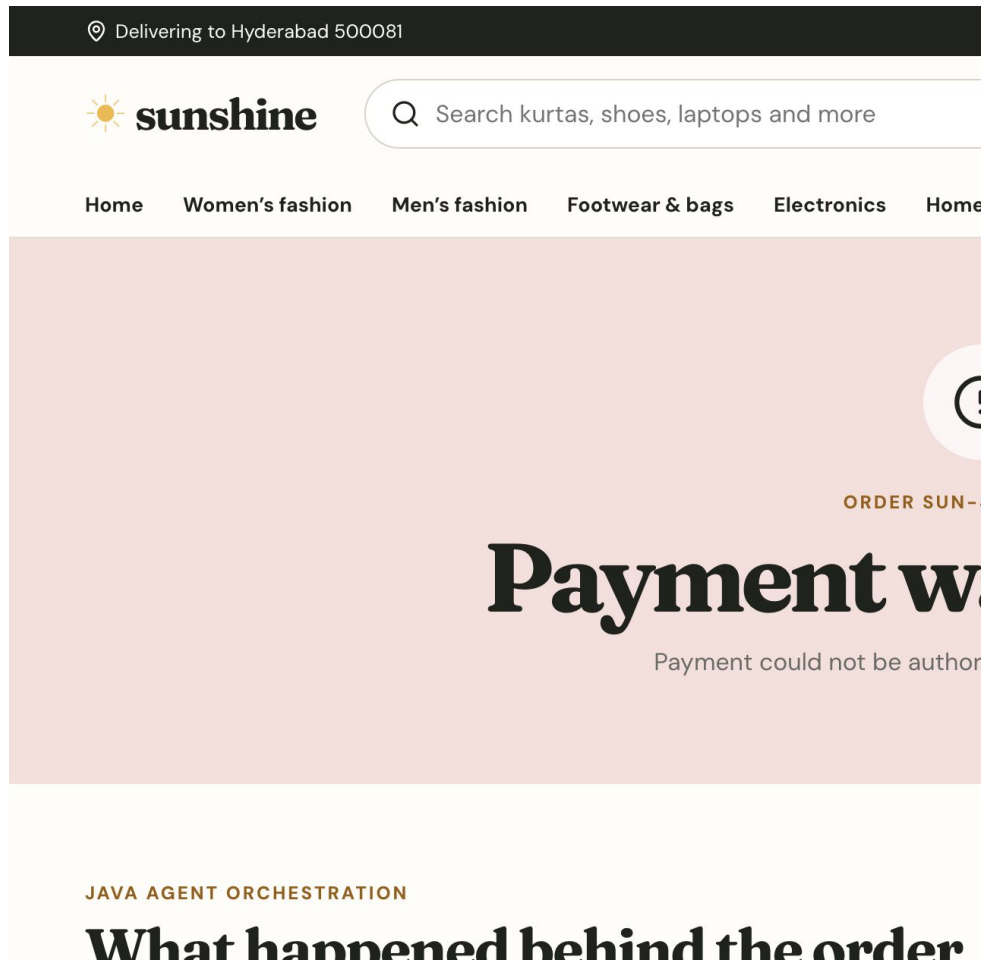


Decision	Reason
Warm cream, yellow and green palette	Creates a recognisable Sunshine identity without imitating Amazon
Server-rendered catalogue pages	Fast initial content and shareable product URLs
Small client components	Interactivity is limited to cart, size, checkout and recommendations
CSS product illustrations	Avoids unlicensed brand photography and keeps the repository self-contained
Responsive grids	Five desktop columns become two on small phones

Accessibility checks include labelled search, semantic navigation, form labels, radio groups, keyboard-reachable buttons, visible focus styles and readable success/failure text that does not depend only on colour.

A failed payment is a designed outcome

A portfolio system should explain what happens when an operation does not succeed, not hide every failure behind a generic error message.



Observed behaviour

- The catalogue reservation completes before payment begins.
- The Payment Agent returns failed with a method-specific explanation.
- The Fulfilment Agent is marked skipped because payment eligibility was not met.
- No payment reference or delivery date is generated.
- The cart remains available so the customer can retry without rebuilding it.

This trace turns a failure into evidence of control flow, state handling and customer communication.

Automated checks across the complete stack

Tests target business behaviour rather than only verifying that files compile.

Quality gate	Coverage	Result
Vitest	Pricing, delivery fee and three adapter scenarios	6 passed
JUnit	Success, payment decline and out-of-stock orchestration	3 passed
pytest	Ranking category, determinism, API and validation	4 passed
ESLint	React and TypeScript quality rules	Passed
Next.js build	Routes, types and 50 product paths	59 pages generated
Browser QA	Product, cart, checkout, success and failure	Passed
Integration	Next.js -> Java and Next.js -> Python	Both sources confirmed

Continuous integration

GitHub Actions runs three independent jobs. The frontend job installs Node 22 dependencies, tests, lints and builds. The Java job sets up Temurin 17 and runs Maven Wrapper tests. The Python job installs versioned requirements and runs pytest.

Deployment

- Docker Compose starts the Next.js, Java and Python services as one local system.
- Vercel hosts the public Next.js interface and compatible route adapters.
- Environment variables switch the route handlers to external Java and Python services without browser changes.

Environment variable	Purpose
JAVA_BACKEND_URL	Proxy order requests to Spring Boot
PYTHON_BACKEND_URL	Proxy recommendation scoring to FastAPI

Security boundary

The demo never requests real card, UPI, password or customer identity data. The checkout form explicitly asks for sample information and does not persist delivery details.

What this project proves and what comes next

The strongest part of Sunshine is not any single framework; it is the connection between customer state, service contracts, failure rules and verifiable outputs.

Skills demonstrated

Interview topic	Concrete example to discuss
React state	Versioned cart store built with useSyncExternalStore
Next.js	Server-rendered product paths plus small client boundaries
Java design	Constructor-injected agents coordinated by OrderOrchestrator
Python API	Typed FastAPI request model and deterministic ranking
Data analysis	CSV-to-JSON KPI pipeline with pandas
Failure handling	Early exit, skipped steps and retry-safe cart state
Deployment	Contract-preserving adapter for platform runtime limits

Limitations

- Synthetic data cannot prove real customer demand or production-scale performance.
- The application has no authentication, persistent order database or live stock source.
- Content-based ranking does not learn from user behaviour.
- Illustrative agent durations are trace labels, not benchmarks.

Reasonable next steps

- Add PostgreSQL order persistence and idempotency keys.
- Deploy Spring Boot and FastAPI to suitable container runtimes and configure Vercel URLs.
- Introduce a real event log and observable correlation ID across services.
- Replace synthetic recommendations only when consented behavioural data is available.

Sunshine is intentionally a finished, explainable student project rather than an unfinished enterprise platform.

Divya Rachala | github.com/divyarachala1812